MEDIUM-2：计算机中的整数与浮点数

Step1：甜度与温控传感器 —— 补码与类型隐式转换

【任务 1.1：到底有多甜？】

`1001 1100`

1.如果该寄存器按无符号整数解析，这杯奶茶的甜度值是多少？

`8+4+16+128=156` 甜度值是 `156`

2.如果老板要求该寄存器按有符号补码整数解析（负数表示少糖），此时的甜度值是多少？

`-128+4+8+16=-100` 甜度值是`-100`

3.那么问题来了：为什么现在的计算机底层清一色都采用“补码”来存储有符号数，而不是直观的“原码”或“反码”？

① 补码能将减法运算转化为加法运算，CPU 只需要实现加法器，简化硬件电路设计； ② 消除了原码、反码里 “正 0、负 0” 两种不同编码表示 0 的歧义，充分利用二进制编码空间；（原来原码的`10000000`不再代表 - 0，拿来表示-128） ③ 相比原码、反码，补码运算不需要额外修正操作，运算逻辑更简单。

【任务 1.2：冰块引发的火灾警报】

输出：**[警告!] 温度过高！触发消防喷淋系统！**

有符号数与无符号数进行比较时的隐式类型转换规则：**有符号整数和无符号整数做比较运算时，有符号数会被隐式强制转换成无符号数，再比较。**

所以，`-10`存储在`32位`的int中经过强制转换会变成一个近似于`2的32次`的数，绝对远大于`50`。

Step2：反物质珍珠 —— 整数溢出与二进制下的运算

【任务 2.1：算术溢出的惨剧】

`01011000` `01000001`

这两个 8 位补码相加后，寄存器中实际保存的二进制位模式是多少？

`10011001`

转换为十进制后，系统认为杯子里现在有多少颗珍珠？这是正溢出还是负溢出？

`-128+8+16+1=-103` **正溢出**

为了防止加法溢出导致奶茶机死机，请用 C 语言补全以下检测函数。

判断最高位是否为1，

只有两个数符号相同才有可能溢出；一正一负相加，结果一定在范围内，不会溢出。

两个正数（`x_sign=0, y_sign=0`）相加，结果变成负数 `sum_sign=1` **正溢出**

两个负数（`x_sign=1, y_sign=1`）相加，结果变成正数 `sum_sign=0` → **负溢出**

```
 // 若相加不发生溢出返回 1，发生溢出返回 0
int is_safe(int x, int y) {

    int sum = x + y;
    int x_sign = (x >> 31) & 1;
    int y_sign = (y >> 31) & 1;
    int sum_sign = (sum >> 31) & 1;
    int rusult;

    // 请补全代码写在TO DO与END OD TO DO之间:
    //TO DO
    if(x_sign==y_sign){
        if(x_sign!=sum_sign){
            result=0;
        }
    }
    result=1;

    //END OF TO DO

    return  result;
}
```

【任务 2.2：没有乘法器的悲哀】

`y = x * 29 = x * 32 - x * 3`

`x * 3 = x * 4 - x`

最终 `y = x * 32 - x * 4 + x`

```
int final_milk(int x) {


    int y;

    // 请补全代码写在TO DO与END OD TO DO之间:
    //TO DO
    y = (x << 5) - (x << 2) + x


    //END OF TO DO

    return y;
}
```

Step3：丝滑配比阀 —— 认识IEEE 754标准下的浮点数

【任务 3.1：对学长提示的深入理解】

1.float（单精度浮点数）与 double（双精度浮点数）各部分位数

类型	总位数	符号位 sign	阶码 exponent	尾数 significand
float（单精度）	32 bit	1 bit	8 bit	23 bit
double（双精度）	64 bit	1 bit	11 bit	52 bit

2.阶码偏置 Bias

偏置计算公式

`Bias = 2^(k-1)-1`

k = 阶码所占比特位数。

- float：阶码 8 bit，(Bias = 127)
- double：阶码 11 bit，(Bias 1023)

> 真实阶码 E，存储在内存中的阶码字段 (e = E + Bias)。

使用偏置的意义

1. 将**有符号的阶码**转换为**无符号整数**存储，硬件比较电路可以直接按无符号整数大小比较浮点数，不用额外处理符号，简化硬件设计。
2. 保证规格化数存储的阶码字段，数值范围连续，方便排序。

---

3. **非规格化数**的特点是什么?

当阶码字段**全部为 0**时，代表非规格化数：

1. 不再隐含整数位`1`，有效数字是`0.xxxxx`。
2. 作用：**填补规格化数到 0 之间的空隙**，实现**逐渐下溢**，可以表示非常接近 0 的小数，避免规格化数最小正数直接跳到 0 的突变。
3. 阶码固定使用 (E=1-Bias)，不再使用存储的阶码值。
4. 可以表示正负 0（符号位不同，其余全 0）。

4.此外，还有两个**特殊阶码**模式：

1. 阶码全为 1 且尾数全为 0

> 表示无穷大

1. 阶码全为 1 且尾数非 0

> 表示无定义数

【任务 3.2：补全浮点数对照表】

二进制位模式	编码类型 (规格化 / 非规格化 / 特殊值)	实际阶码 e (十进制)	尾数真值 M (二进制小数)	十进制最终真值
0 0110 100	规格化	6−7=−1	1.1002=1.5	1.5×2−1=0.75
0 0000 011	**非规格化**	**1-7=-6**	**0.011=0.375**	**0.375*2的-6次=0.005859375**
0 1111 000	**特殊值**	不适用	不适用	+∞ (正无穷)
0 1001 010	规格化	**9-7=2**	**1.01=1.25**	**1.25×2×2=5**

【任务 3.3：阀门的极限】

1.该 8 位浮点数能表示的最小的正非规格化数的二进制位模式是什么？对应的十进制数值是多少？最大的呢?

最小`0 0000 001` `0.001=0.125` 十进制为`0.125×2^(-6)次=0.001953125`

最大 `0 0000 111` 同理 十进制为`0.013671875`

2.阀门全开时，该 8 位浮点数能表示的最大的正规格化数的二进制位模式是什么？对应的十进制数值是多少？最小的呢？

最大 `0 1110 111` `1.111=1.875 14-7=7` 十进制为`1.875×2^7=240`

最小 `0 0000 001` `1.000=1. 1-7=-6` 十进制为`1×2^-6=0.015625`

3.比较最大正非规格化数和最小正规格化数，思考对于非规格化形式，阶码值为什么是1−Bias而不是简单的−Bias？

1. 弥补非规格化数没有隐含前导 1 造成的"数值减半"；
2. 让最大非规格化数与最小规格化数连续衔接；
3. 实现渐进下溢，使浮点数在接近 0 时不会突然断档。

> 如果规定非规格化数的实际阶码为： 0-bias = −7
>
> 那么最大正非规格化数会变成`0.0068359375`，与`0.015625`有空隙

Step4：消失的营业额 —— 浮点加法、舍入与结合律

【任务 4.1：设计加法程序】

Debug记录

1. 输入要用 char 数组，不能用 int 直接读

输入是`01001011`这样的**二进制字符串**，如果用`int`配合`%d`读取：

- 会被当成十进制数解析，前导 0 丢失，数值完全错误
- 无法对应原始的 8 位二进制位模式 char 数组可以逐位保存每个 '0'/'1' 字符，再通过`-'0'`转为整数 0/1，精准对应每一位。

2. int 变量直接二进制移位

int 在内存中永远以二进制存储，`>>`和`<<`直接操作内存中的二进制位，和代码里用十进制还是十六进制书写无关。 例如`int sig = 11`，内存中就是`1011`，`sig >> 1`就是把二进制位右移一位得到`101`（十进制 5），正好对应尾数右移、向零舍入的效果。

3. 不需要考虑左规格化了

两个非负规格化数相加：

- 大阶码的尾数最小是 8（对应 1.000）
- 小阶码的尾数右移后最小是 0

- 相加结果`sum_sig >= 8`恒成立 因此只会出现进位导致的右规，不会出现尾数不足需要左移的情况。

4. 整数 0/1 和字符 '0'/'1' 混淆

不能直接拿`(weishu%2)`类似的式子直接赋值给`c[q]`

- 整数 0/1：数值，用于运算、移位
- 字符 '0'/'1'：ASCII 码分别是 48 和 49，用于字符串打印 输出时必须把整数位转换为对应字符，否则`printf("%s")`会打印出不可见控制字符。\

> 技巧：数字 ？ ‘1’ ： ‘0’

完整代码

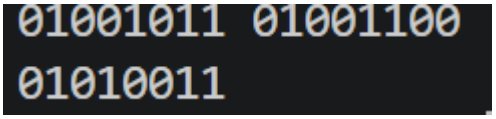
```c
#include <stdio.h>

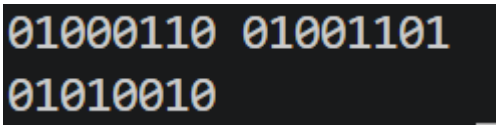
char a[9];
char b[9];
char c[9];

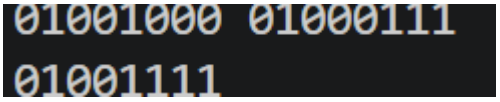
int main(){
    scanf("%s %s",a,b);
    //第1 2 3 4位是阶码
    int jiema1=0,jiema2=0;
    for(int i=1;i<5;i++){
        jiema1=(a[i]-'0')+jiema1*2;//存进去的是字符，无法直接加减
    }
    for(int j=1;j<5;j++){
        jiema2=(b[j]-'0')+jiema2*2;//存进去的是字符，无法直接加减
    }
    int jiema;
    int jiemacha;
    if(jiema1>jiema2){
        jiema=jiema1;
        jiemacha=jiema1-jiema2;
    }
    else{
        jiema=jiema2;
        jiemacha=jiema2-jiema1;
    }
    int weishu1=0,weishu2=0;
    for(int i=5;i<8;i++){
        weishu1=(a[i]-'0')+weishu1*2;
    }
    for(int j=5;j<8;j++){
        weishu2=(b[j]-'0')+weishu2*2;
    }
    weishu1=8+weishu1;//补1
    weishu2=8+weishu2;
    weishu1=weishu1>>(jiema-jiema1);
    weishu2=weishu2>>(jiema-jiema2);
    int weishu=weishu1+weishu2;
    while(weishu>=16){
        weishu = weishu >> 1;
        jiema++;
    }
    for(int k=7;k>4;k--){
        c[k]=(weishu%2) ? '1' : '0';
        weishu=weishu/2;
    }
    for(int q=4;q>0;q--){
        c[q]=(jiema%2) ? '1' : '0';
```

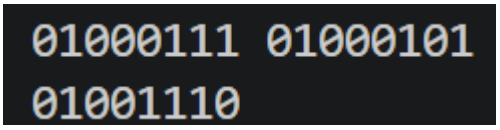
```
        jiema=jiema/2;
    }
    c[0]='0';
    c[8]='\0';
    printf("%s",c);
    return 0;
}
```

运行结果









【任务 4.2：老板为什么每天都在亏钱？】

请你从 IEEE 754 浮点数的对阶机制、精度丢失与大数吸收的角度向老板解释：为什么同样的三个数，只是加的顺序变了，那 15 块钱就凭空消失了？（即为什么浮点数加法不满足数学上的结合律）。

1. **浮点数不是实数，只是有限离散集合**，每一步运算后都要舍入到最近可表示浮点数，不是精确数学运算。
2. **加法需要对阶**：大数加小数时，小数的有效位被右移丢弃，小数直接被 “吃掉”。
3. 不同运算顺序，**舍入发生在不同步骤**：
    - ((a+b)+c)：先把很小的a加到很大的b上 → a丢失
    - (a+(b+c))：先把(b,c)抵消归零，再加a，a保留下来